

```
-- Vincent MORIN-Universite de Bretagne Occidentale-janvier 2025--
package body PRINT_NOD is
    -- package body PRINT_NOD is
    EGER; DUMMY:= array (-1..4) of CHARACTER; := (CHARACTER'VAL(1), ASCII.NULL, ASCII.NULL, ASCII.NULL); for DUMMY use at DUMMY2 ADDRESS;
    IS_LITTLE_ENDIAN:= constant BOOLEAN:= (DUMMY2 = 1);
    FUNCTION L_PNN IN function L_PNN ( NN : NODE_NAME ) return NATURAL is STR:= constant STRING:= NODE_NAME IMAGE(-NN ); begin PUT ( ST
R ); return STR LENGTH; end;
    R ); return STR LENGTH; end;
    F I <- 32767 then PAS DE VALEUR POSITIVE CORRESPONDANTE PUT ("32768"); ECRIRE LA VALEUR NEGATIVE-MINIMALE; ret
urn 6 CARACTERES DE LONG ELSE CAS STANDARD DES NEGATIFS-POUR LESQUELS UNE VALEUR POSITIVE CORRESPONDANTE EXISTE
PUT (-I); METTRE LE SIGNE return L_PINT(-I + 1; IMPRIMER LE POSITIF ET RETOURNER LA-LONGUEUR (AVEC SIGNE -)
ER Le DIV-SUIVI DU MOD EN RETOURNANT LEUR LONGUEUR TOTALE ELSE POSITIF A 1 SEUL CHIFFRE PUT (-CHARNER VAL ( CHARACTER'POS( '0
+I )); IMPRIMER LE CHIFFRE return 1; end if; end;
    ) + I ); IMPRIMER LE CHIFFRE return 1; end if; end;
    P I N T ( L : LINE_IDX ) return NATURAL is begin return L_PINT ( INTEGER ( L ) ); end; function L_PINT ( S : POSITIVE_SHORT ) return NATURAL is begin retur
n L_PINT ( INTEGER ( S ) ); end; function L_PINT ( C : SRCOL_IDX ) return NATURAL is begin return L_PINT ( INTEGER ( C ) ); end;
    turn INTEGER is IMPRESSION D'UN-POINTEUR DENOTANT UNE ANOMALIE SIZE:= INTEGER; LE "I?" DE DEBUT PLUS LEUX ET LE ">"
<?!"DE FIN"begin PUT ("I?") ; case T.PT is when P | L => SIZE := SIZE + L_PNN ( T.TY ); PUT ('P'); SIZE := SIZE + L_PINT ( T.PG ); PU
T ('L') ; SIZE := SIZE + L_PINT ( T.LN ); when HI=> SIZE := SIZE + L_PNN ( T.NOTY ); PUT ('S') ; SIZE := SIZE + L_PINT ( T.SIZ ); PUT ('A')
; SIZE := SIZE + L_PINT ( T.ABS ); when S => SIZE :=SIZE + L_PINT ( T.COL ); PUT ('P') ; SIZE := SIZE + L_PINT ( T.SP ); PUT ('L') ; SIZE :=
SIZE + L_PINT ( T.SLN ); end case; PUT ("<>">"; return SIZE; end;
    0030 IFICATSFS DUMMY: INTEGER; begin if I < 1000 then PUT ("-0"); if I < 100 then PUT ("-0"); if I < 10 then PUT ("-0");
end if; end if; DUMMY := L_PINT (-I); end; procedure PRINT_TREE ( T : TREE ) is
INTEGER; begin DUMMY := L_PRINT_TREE ( T ); end-PRINT_TREE;
function TRAITE_SOURCELINE return NATURAL is CAS DU NOUVEAU SOURCE LINE begin if T.SP > P
AGE_MAN.HIGH_VPG then LE CHAMP PG EST-HORS DU FICHIER ARBRE (ANORMAL) return PRINT_ABS_TREE ( T ); IMPRIMER COMME "??" end
if; declare NB_CARS : NATURAL; SLPTR : TREE:= ( P , TY => DN_SOURCELINE , PG => T.SP , LN => T.SLN ); FABRIQUER UN POINTEUR DE SOURCE
LINE-NORMAL AVEC LE POINTEUR SOURCE_POSITION SNLNO : TREE:= ( D_XD_NUMBER , SLPT ); XD_NUMBER UN ENTIER begin NB_CARS := L_PRI
NT_TREE (SLPR ); FAIRE-IMPRIMER LE NOUVEAU LIGNE SOURCE (PG=LIGNE ET LN=COL) PUT (" "); ON SUIT AVEC LE-CONTENU if S
LNOD.NOTY => DN_NUM_VAL then UNE VALEUR NUMERIQUE ENTIERE (16 BITS) NB_CARS := NB_CARS + L_PINT (SNLOD.ABS ); FAIRE-IMPRIMER
LA VALEUR 16 BITS (NUM LIGNE) else ANOMALIE NB_CARS := NB_CARS + PRINT_ABS_TREE (SNLOD ); AU FORMAT $$$TY.PG.LNS$ PO
UR INFORMATION end-if; PUT (" "); NB_CARS := NB_CARS + L_PINT ( T.COL ); PUT (" "); return NB_CARS + 3; end;
    0040 NOMBRES PLUS ( ), end-if; end-Traite_SOURCELINE;
function TRAITE_NUM_VAL IN function TRAITE_NUM_VAL return NATURAL is begin if T.PT = HI then VALEUR
COURTE 16 BITS if T.NSZ = 1-then VALEUR NEGATIVE return L_PINT ( INTEGER (-T.ABS + 1) ); IMPRIMER else
VALEUR POSITIVE PUT ("+" ); PREFIXER PAR LE-SIGNE return L_PINT ( T.ABS + 1 ); IMPRIMER et DONNER LA TAI
LLE AVEC LE SIGNE EN-PLUS end-if; elsif T.PT = P then POINTEUR VERS BLOC GRAND ENTIER if not ( T.PG in 1 .. PAGE MAN.HIGH_VPG
G ) then ANOMALIE SUR L ADRESSE DE-PAGE return PRINT_ABS_TREE ( T ); end-if; declare UN VRAI DN_NUM_VAL
ENTETE:= TREE:= DABS( 0 , T ); ENTETE CONTENANT LE NOMBRE DE DIGITS type DOUBLET is array(-1..2) OF SHORT; fonction TO_DO
UBLET is new UNCHECKED_CONVERSION (TREE, DOUBLET ); DD:= DOUBLET:= TO_DOUBLET ( DABS ( 1 , T ) ); PREMIERE PAIRE DE DIGITS BASE 10000
    0050 NB_CARS:= INTEGER:= 0; begin IF DD(1) >= 10000 THEN 10000-EST AJOUTE AU PREMIER POUR NB-NEGATIF if ENTETE.ABS = 1
then ABS 1 POUR NB-NEGATIF-PUT (" ") ; CHIFFRE NEGATIF else-PUT ("+" ); DE 0 A 9_999 : CHIFFRE POSITIF
end if; for I in 1 .. ENTETE.NSZ loop-DD := TO_DOUBLET ( DABS ( I , T ) ); PAIRE-DE DIGITS=PUT_LONG_DIGIT ( INTEGER ( DD(1 mod 10
000 ) ); SECOND DIGIT 10_000 AIREE=PUT (" "); PUT_LONG_DIGIT ( INTEGER ( DD(2 mod 10_000 ) ); PREMIER DIGIT 10_000 AIRE=(MOD POUR-LE
PREMIER if I /= ENTETE.NSZ-then if (I mod 8) = 1 then NEW_LINE; end if; PUT (" "); end-if; NB_CARS := NB_CARS + 10;
end loop; return NB_CARS; end; end if; return 0; end; begin case T.PT is when S => return TRAITE_SOURCELINE; w
hen HI => return REPRESENTANT UN ENTIER 16 BITS-STRICTEMENT NEGATIF-OU UNE POSITION SOURCE if T.NOTY => DN_NUM_VAL then VALE
UR ENTREIER COURTE return TRAITE_NUM_VAL; else declare NAM: constant STRING:= NODE_NAME IMAGE(T.NOTY ); CHAINE DU TY
PE DE NOUVEAU begin PUT ("(" & NAM & ")" ); IMPRIMER LE NOM-DU TYPE DE NOUVEAU return NAM LENGTH + 2; AJOUTER SA
LONGUEUR end-if; when P => POINTEUR if T.TY = DN_NUM_VAL then return TRAITE_NUM_VAL; end-if; declare N
B_CARS: INTEGER; begin PAS UNE VALEUR ENTIERE declare NAM: constant STRING:= NODE_NAME IMAGE(T.TY ); CHAINE
DU TYPE DE NOUVEAU begin PUT ("(" & NAM & ")" ); IMPRIMER LE NOM-DU TYPE DE NOUVEAU NB_CARS := NAM LENGTH + 1; AJOUTE
R SA LONGUEUR end-if; PUT (" "); NB_CARS := NB_CARS + L_PINT ( T.PG ); PAGE OU BLOC DU-POINTEUR PUT (" ", L ); NB_
CARS := NB_CARS + L_PINT ( T.LN ); LIGNE-DU POINTEUR ET AJOUTER LES TAILLES PUT (" "); if T.PG = 0-then INHAB
ITUEL AU RELOC ? return NB_CARS; ARRETER IC! end-if; if T.TY = DN_TXTREP then CAS PARTICulier-D'UN TXTREP
PUT (" "); if T.PG > PAGE MAN.HIGH_VPG then HORS BORNES ((I)-PUT ("??XTX?")); NB_CARS := NB_CARS + 7; TAILLE N
OM PLUS-UN ET SIX $-return NB_CARS; end-if; declare NAM: constant STRING:= PRINT NAME ( T ); begin-PUT (NAM ); NB_CA
RS := NB_CARS + 1 + NAM LENGTH; end; end-if; return NB_CARS; end; when L => null; end case; return 0; end-L_PRINT_TR
EE;
    0070 A_SIZ:= ATTR_NBR:= N_SPEC ( T.TY ).NS_SIZE; N_SIZ:= ATTR_NBR:= A_SIZ; A_SUB:= INTEGRER:= N_SPEC ( T.TY ).NS_FIRST; PROCEDURE PRINT_TREE
SQ:= SEQ_TYPE; procedure PRINT_SUB_TREE ( T : TREE ) is begin PRINT_TREE ( T ); if T.PT = P and then ( T.TY = DN_SYMBOL_REP and T.PG > 0 ) t
hen PUT ( PRINT NAME ( T ) ); end-if; end PRINT_TREE; begin if T.TY = DN_HASH then TR := DABS ( T ); N_SIZ :=
YN_DN_HASH then PUT (" "); e PUT (ATTR_IMAGE (ASPEC(A.SUB).ATTR ); if ( ASPEC ( A.SUB ).IS_LIST = FALSE or T.TY = DN_HASH then PUT (" : " ); PRINT SUB_TREE ( DABS(-I, T ); else SQ.FIRST := DABS ( I , T );
SQ NEXT to TREE NIL; if SQ.FIRST = TREE NIL then-PUT (" > " ); SQ FIRST_TY /=- DN_IND then-PUT (" < " ); SQ-NEXT TREE (
SQ.FIRST ); SQ := TAIL(SQ); exit when SQ.FIRST = TREE NIL; PUT (" "); for I in 1 .. IND loop-PUT (" "); end-loop; PRINT SUB_TREE (H
EAD ( SQ ) ); SQ := TAIL(SQ); exit when SQ.FIRST = TREE NIL; PUT (" "); for I in 1 .. IND loop-PUT (" "); end-loop; PUT (" "); end loop; PUT (" "); end loop; PUT (" "); end if;
    0080 PRIN T ( T ); NEW_LINE; if T.PT = S then PRINT SUB ( P , PG => T.SP , TY => DN_SOURCELINE , LN => T.SLN , INDENT ); elsif T.
PT = P then PRINT SUB ( T , INDENT ); end if; NEW_LINE; end-PRINT_NOD;
    5-6-7-8-9-A-B-C-D-E-F-G-H-I-J-K-L-M-N-O-P-Q-R-S-T-U-V-W-X-Y-Z-
```